14 tidyverse数据整形与连接

2026/3/31

写在前面：暂无。

修改列名

- 使用函数`rename()`来修改列名：

```
data %>% rename(PatientID = PID) # 修改前名称为"PID"，修改后名称为"PatientID"
```

- 常用的场景是，你有若干不同来源的数据，这些数据各列的名称可能各不相同，你要先同一列名，再把数据合并到一起。

合并数据

- 有两种合并方式，一种是纵向合并（按照行来合并），一种是横向合并（按照列来合并）。
- 对于纵向合并，
  - 可以使用`tidyverse`的`bind_rows()`函数进行合并：

```
bind_rows(<待合并数据框1>, <待合并数据框2>)
```

  - 也可以使用base `R`中的函数进行合并：

```
rbind(<待合并数据框1>, <待合并数据框2>)
```

- 对于横向合并，
  - 可以使用`tidyverse`的`left_join()`函数进行合并。"left"的意思是，传入参数里左侧的那个数据是“主数据”，右侧数据要对齐到“主数据”(比如，右侧数据有多余的、主数据中没有的部分，则丢弃)：

```
left_join(<待合并数据框1>, <待合并数据框2>, by = <合并依据>)
data %>% left_join(data_new2, by = "PID") # 举个例子
```

  - 对于base `R`，则有三种方法：
    - 两个数据框行数一样、顺序完全一样，只是想把列拼在一起，则可以直接使用`$`。优点是简单、快捷，缺点是没有按ID匹配，只是硬拼，顺序乱了就全错：

```r
df1 <- data.frame(a = 1:3, b = 4:6)
df2 <- data.frame(c = 7:9, d = 10:12)

# 横向合并（直接追加）
df1$c <- df2$c
df1$d <- df2$d
```

- 使用 `cbind()` 函数实现基础横向合并，但其实逻辑和 `$=` 一样，只是按位置硬拼，不匹配ID：

```r
df_new <- cbind(df1, df2)
```

- 使用 `merge()` 函数，按ID对齐合并：

```r
# 左连接（以df1为准，匹配df2）
df_new <- merge(df1, df2, by = "ID", all.x = TRUE)

# 全连接（保留所有行）
df_new <- merge(df1, df2, by = "ID", all = TRUE)
```

- 多表连接：可以用管道符将多个 `left_join()` 串起来，这样就可以拼出来一张更大的表。是一种数据整合/数据清洗的方式。

补充

- UP最后补充说，使用 `tidyverse` 的时候，新增数据框最好使用 `tibble` 格式，而不要用 `data.frame` 。可以用这个函数将 `data.frame` 转换为 `tibble` ：

```r
# 假设你有一个普通 data.frame
df <- data.frame(a = 1:3, b = letters[1:3])

# 转换为 tibble
tb <- as_tibble(df)
```

- `tibble` 相比 `data.frame` 的优点：
  i. 打印输出: `data.frame` 会把所有行、所有列都打印出来，数据大了直接刷屏； `tibble` 只打印前10行+能放下的列，还显示每列的数据类型；
  ii. 取列行为： `data.frame` 如果列名不存在，会返回 `NULL` ，不报错，容易出bug； `tibble` 如果列名不存在，直接报错+警告；
  iii. 列名自动修复：列名非法时， `data.frame` 会乱改列名； `tibble` 不自动修改列名，非语法列名也能保留（用反引号调用）；
  iv. `tibble` 支持更灵活的数据类型，允许列里放列表、数据框、函数、复杂对象等；
  v. `tibble` 不自动把字符串转成因子，而 `data.frame` 默认把字符串变成因子。